Rapport de Validation

Projet Génie Logiciel

Équipe g125
Alexandre Danhiez, Noah Deplace, Ilès Ouyahia,
Nicolas Montauriol, Nathan Vigouroux
Ensimag

Janvier 2026

Table des matières

1 Descriptif des tests **2**
- 1.1 Analyse Syntaxique (Étape A) . . . 2
- 1.2 Analyse Contextuelle (Étape B) . . . 3
- 1.3 Génération de code (Étape C) . . . 3

2 Scripts d'exécution **4**
- 2.1 Fonctionnement . . . 4
- 2.2 Manuel d'utilisation . . . 4
- 2.3 Tests de non-régression . . . 5

3 Gestion des risques **5**

4 Gestion des rendus **5**
- 4.1 Rapports et Documents . . . 5
- 4.2 Séances de suivi . . . 6
- 4.3 Rendus . . . 6

5 Couverture des tests (JaCoCo) **6**
- 5.1 Utilisation et Interprétation . . . 6
- 5.2 Analyse des résultats . . . 7

6 Autres méthodes de validation **7**

Appendices **7**

A Risques Organisationnels **8**

B Risques Techniques **9**

1 Descriptif des tests

Afin de garantir la fiabilité et la robustesse de notre compilateur, nous avons mis en œuvre une stratégie de validation complète couvrant les trois étapes majeures du projet : l'analyse lexico-syntaxique (Étape A), la construction de l'arbre abstrait et les vérifications contextuelles (Étape B), ainsi que la génération de code (Étape C).

Notre approche repose sur différents types de tests :

— **Tests unitaires :** Développés en Java avec *JUnit* et *Mockito*, ils permettent de tester les méthodes de manière isolée, garantissant le bon fonctionnement des méthodes et classes du compilateur deca.

— **Tests système :** Ces tests consistent à compiler et exécuter des fichiers sources `.deca`. Ils assurent que la chaîne complète de compilation fonctionne correctement, que ce soit pour le compilateur dans son entièreté, mais également pour chaque étape du projet.

— **Tests de non-régression :** Intégrés à notre pipeline sur Git, ces tests sont lancés à chaque ajout de code. Ils permettent de vérifier qu'une nouvelle fonctionnalité n'altère pas le comportement des fonctionnalités existantes et que le code ajouté ne compromet pas le reste du travail.

— **Tests « Black box » :** Tests réalisés « à l'aveugle » sur des fonctionnalités, souvent écrits avant même que le code ne soit implémenté ou sans consulter l'implémentation interne. Réalisés en majorité par les membre de l'équipe n'ayant pas participé au codage du compilateur.

— **Tests « Glass box » :** Tests structurels visant à valider des fonctionnalités en se basant sur la structure interne du code source.

Les parties suivantes détaillent l'organisation de ces tests pour chaque étape.

1.1 Analyse Syntaxique (Étape A)

L'organisation des tests pour l'étape A est structurée comme suit :

```
deca/
└── syntax/
    ├── valid/ ... Fichiers .deca valides (sont corrects pour l'étape A)
    │   ├── created/ ... Tests créés par l'équipe
    │   │   ├── objet/ ... Tests portant sur le langage deca final
    │   │   └── sans-objet/ ... Tests portant sur le langage deca sans objet
    │   └── provided/ ... Tests fournis
    └── invalid/ ... Fichiers .deca invalides (doivent échouer)
        ├── created/ ... Tests créés par l'équipe
        │   ├── objet/ ... Tests portant sur le langage deca final
        │   └── sans-objet/ ... Tests portant sur le langage deca sans objet
        └── provided/ ... Tests fournis
```

Ces dossiers contiennent les **tests système black-box** écrits en langage Deca. Ils vérifient que le compilateur accepte les programmes valides syntaxiquement et rejette ceux qui ne le sont pas.

Parallèlement, des **tests unitaires Glass Box** sont situés dans :

`java/fr/ensimag/deca/syntax/`

Une distinction est faite entre les tests ciblant le langage objet et le langage sans objet.

Cette séparation nous permet de séparer les règles de grammaire spécifiques. Par exemple, nous vérifions indépendamment que les structures de base (boucles `while`, conditions `if`) sont

correctement parsées avant de valider des structures plus complexes comme la déclaration de classes ou l'appel de méthodes. De plus, des tests spécifiques ciblent le *Lexer* pour s'assurer que chaque token (identificateurs, chaînes de caractères, commentaires) est bien reconnu avant même l'étape d'analyse syntaxique.

1.2 Analyse Contextuelle (Étape B)

L'organisation pour l'étape B suit une logique similaire :

```
deca/
└── context/
    ├── valid/ ... Fichiers .deca valides (sont corrects pour l'étape A et B)
    │   ├── created/ ... Tests créés par l'équipe
    │   │   ├── objet/ ... Tests portant sur le langage deca final
    │   │   └── sans-objet/ ... Tests portant sur le langage deca sans objet
    │   └── provided/ ... Tests fournis
    └── invalid/ ... Fichiers .deca invalides (doivent échouer)
        ├── created/ ... Tests créés par l'équipe
        │   ├── objet/ ... Tests portant sur le langage deca final
        │   └── sans-objet/ ... Tests portant sur le langage deca sans objet
        └── provided/ ... Tests fournis
```

Ces fichiers constituent nos **tests système** réalisés en **Black-box**. L'objectif est de valider le respect de l'intégralité des règles contextuelles (typage, portée des variables, signatures de méthodes). Nous nous assurons ainsi que tout programme sémantiquement invalide (par exemple : utilisation d'une variable non déclarée, redéfinition interdite, ou opérations entre types incompatibles) est systématiquement rejeté avec un message d'erreur explicite, conformément au polycopié de spécification.

De nombreux **tests unitaires** ont également été réalisés en Java afin d'assurer, par exemple, les contraintes de typage des règles portant sur les opérations.

Situés dans `java/fr/ensimag/deca/context/`, ils combinent approches Black box et Glass box (avec Jacoco) pour maximiser la couverture de code.

Ces tests unitaires se focalisent notamment sur :

— **La compatibilité des types :** Validation des types pour les opérations binaires et unaires, incluant les conversions implicites (ex : `int` vers `float`) et les sous-typages en mode Objet.

— **Les signatures :** Contrôle de la cohérence des signatures de méthodes lors de la déclaration et de l'appel.

1.3 Génération de code (Étape C)

Pour la génération de code, l'arborescence est légèrement plus développée, puisqu'elle englobe également des tests interactifs, de performance et d'impression :

```
deca/
└── codegen/
    ├── interactive/ ... Fichiers .deca nécessitant une action externe
    ├── perf/ ... Fichiers .deca d'évaluation des performances
    │   └── provided/ ... Tests fournis
    ├── print/ ... Fichiers .deca de validation des méthodes d'impression
    ├── invalid/ ... Fichiers .deca invalides (doivent échouer)
    │   └── created/ ... Tests créés par l'équipe
    │       ├── objet/ ... Tests portant sur le langage deca final
    │       └── sans-objet/ ... Tests portant sur le langage deca sans objet
    └── valid/ ... Fichiers .deca valides (sont corrects pour l'ensemble du compilateur)
```

```
├── provided/ ... Tests fournis
└── created/ ... Tests créés par l'équipe
    ├── objet/ ... Tests portant sur le langage deca final
    └── sans-objet/ ... Tests portant sur le langage deca sans objet
```

Il s'agit ici exclusivement de **tests système black box** en Deca. Nous avons jugé les tests unitaires moins pertinents pour cette étape, privilégiant la validation du comportement du programme assembleur généré via `ima`.

En effet, la validation de cette étape repose sur l'état du code machine produit. Nos scripts de tests automatisent donc la chaîne de compilation et comparent la sortie standard obtenue avec un résultat attendu.

Ces tests permettent de valider des aspects critiques :

— **La robustesse à l'exécution :** Contrôle des erreurs à l'exécution (débordement de pile, division par zéro, ...).

— **L'implémentation Objet :** Validation de l'héritage et du polymorphisme.

2 Scripts d'exécution

2.1 Fonctionnement

L'automatisation des tests repose sur des scripts shell situés dans `src/test/script/`. Ces scripts parcourent les dossiers de tests, invoquent le compilateur pour chacune des étapes et comparent le résultat obtenu avec le résultat attendu (ou vérifient l'absence d'erreur pour les tests valides).

Nous utilisons également **Maven** pour la gestion du projet et l'exécution des tests Java :

— `mvn test` : Compile le projet et lance l'ensemble des tests.

— `mvn verify` : Similaire à `mvn test`, permet également de générer un rapport de couverture JaCoCo.

2.2 Manuel d'utilisation

Pour reproduire intégralement notre validation, l'utilisateur doit se placer à la racine du projet et peut utiliser les commandes suivantes :

Exécution de tous les tests :

```
#mvn test
#mvn verify
```

Exécution par étape spécifique :

```
#./src/test/script/test_etapeA.sh   Syntaxe
#./src/test/script/test_etapeB.sh   Contexte
#./src/test/script/test_etapeC.sh   Generation de code
#./src/test/script/common-tests.sh  Tests fournis par les enseignants
#./src/test/script/test_codegen_print.sh   Methodes d'impression
```

Exécution d'un test particulier :

Il est également possible de tester un programme deca individuellement grâce aux commandes suivantes :

```
#lex <nom_fichier>.deca     Analyse Lexicale
#parse <nom_fichier>.deca   Analyse Syntaxique
#sema <nom_fichier>.deca    Analyse Contextuelle
#decac <nom_fichier>.deca   Compilation
#exec <nom_fichier>.ass     Execution
```

2.3 Tests de non-régression

Ces scripts sont automatiquement déclenchés via notre intégration continue (CI) à chaque tentative de *merge* sur la branche principale. Si un seul test échoue, l'action est bloquée, empêchant ainsi l'introduction de régressions dans la branche principale.

De plus, cette pipeline permet d'assurer la validité des push sur la branche principale en exécutant l'ensemble de la suite de tests.

3 Gestion des risques

L'objectif de la gestion des risques est d'anticiper les dangers potentiels lors du projet et de définir des procédures de résolution immédiates.

L'analyse est structurée autour de deux matrices classant les risques selon leur probabilité et leur gravité :

Risques Organisationnels (voir Annexe A) : Cette section se concentre sur la dynamique d'équipe et la gestion du temps. Elle identifie des menaces critiques purement liées à des erreurs humaines de gestion d'équipe.

Risques Techniques (voir Annexe B) : Cette section aborde les échecs liés au code et à l'intégration. Les risques majeurs sont ici liées aux compétences techniques et aléas des machines de travail.

4 Gestion des rendus

Cette procédure est mise en place pour réduire le plus possible les imprévus lors des rendus/-suivis de ce projet. L'exécution de cette dernière permet alors de confirmer le bon fonctionnement du code du projet et d'assurer la qualité des documents à rendre.

Nous avons donc mis en place des procédures concernant respectivement des séances de suivi et les rendus à l'aide de l'outil Notion (Voir Figure 1 ci-dessous).

Checklist	État	Personne assig...	Date d'...	Priorité	Tests associés	Description
bloc principal {...}	Terminé		01/11/2026	Élevée	empty.deca , empty_main.deca , ecrit0.deca	Le compilateur doit reconnaître
Commentaires complexes OUVRIR	En cours	Nathan Vigouroux	01/11/2026	Élevée		Commentaires de commentaire,
Déclaration de variables simples	En cours	Nathan Vigouroux	01/11/2026	Élevée	declaration_valid.deca , decl_var_correct.deca	Gestion des déclarations de vari
Affectations	Terminé		01/11/2026	Élevée	affect_compatible.deca , affect-incompatible.d	Support des affectations simple
expressions arithmétiques	En cours		01/11/2026	Élevée	arith_full.deca , binary_expr.deca , unary_exp	Support des expressions arithm
int, float	En cours		01/11/2026	Élevée	lit_int_valid.deca , lit_float_valid.deca , fl	Gestion complète des types prin
conversion implicites : int —> float (convFloat)	En cours		01/11/2026	Élevée	affect_compatible.deca , arith_full.deca , flo	Insertion automatique des conv
readInt, readFloat	En cours		01/11/2026	Élevée	ReadInt.deca , ReadFloat.deca , TestReadIntUni	Support des primitives d'entrée
print, println, printx, printlnx	Terminé	Ilès Ouyahia	01/11/2026	Élevée	print.deca , println.deca , print_empty.deca ,	Gestion des instructions d'affich
Analyse lexicale	En cours		01/11/2026	Élevée	chaine_valid.deca , identificateurs_valid.deca	Le lexer doit reconnaître tous les
Analyse syntaxique	Terminé		01/11/2026	Élevée	tous les fichiers dans src/test/deca/syntax/valid	Le parser doit accepter uniquem
Vérification contextuelle	Terminé		01/11/2026	Élevée	tous les fichiers dans src/test/java/fr/ensimag/c	Implémentation des règles de ty
AST décoré	Terminé		01/11/2026	Élevée	affect_compatible.deca , condition.deca , dec	L'arbre de syntaxe doit être enri

FIGURE 1 – Exemple d'utilisation de Notion pour le Rendu Intermédiaire

4.1 Rapports et Documents

La procédure suivante s'applique dès qu'un rendu de document est exigé. Que ce soit un simple planning ou un rapport fastidieux, toutes les étapes doivent être respectées pour limiter les plus possible les erreurs.

— **Conformité du nom :** Chaque titre de document doit être au format suivant : **<gl25_titre.ext>**

— **En-tête du fichier :** Le document possède une en-tête dans laquelle on retrouve les informations suivantes :
  — Le titre du document,
  — Le nom du groupe,
  — La date de la version du document.

— **Relecture :** Le document est relu par tous les membres du groupe de manière à ce qu'il n'y ait aucune faute d'orthographe. Cela permet également de faire consensus sur son contenu.

4.2 Séances de suivi

Cette partie concerne les séances de suivi en présence de l'enseignant tuteur mais s'applique également la soutenance finale.

Les étapes de préparation à l'entretien sont donc les suivantes :

— **Sujets abordés :** Vérifier que tous les sujets indiqués dans la doc seront abordés.

— **Démonstrations :** Vérifier que les démos fonctionnent si elles sont réalisées en temps réel. Assurer la cohérence et la visibilité des captures d'écran si la démo est préparée à l'avance.

— **Documents :** Préparer tous les documents susceptibles de servir lors de la présentation ou des questions.

— **Répartition de la parole :** Répartition de la parole entre les membres de l'équipe équitable et cohérente.

— **Slides :** Les slides sont épurées, travaillées et sans fautes d'orthographe.

4.3 Rendus

Les étapes suivantes sont à effectuer sur plusieurs machines différentes afin de confirmer le fonctionnement du projet sur n'importe quel environnement et pas uniquement un ordinateur personnel.

— **decac :** Vérifier que toutes les options de la commande decac sont DISPONIBLES et FONCTIONNELLES (p.103 de la spécification). Prendre en compte la compatibilité des commandes et autres contraintes.

— **Messages d'erreurs :** Vérifier la conformité des messages d'erreurs à la spécification (p.104) au format suivant :
  <nom de fichier.deca> :<ligne> :<colonne> :<description informelle du problème>

— **Organisation :** Vérifier que chaque fichier est au bon emplacement.

— **Compilation :** Assurer la compilation globale du code.

— **Tests :** Vérifier la compilation, l'emplacement et la validité des tests d'acceptance (notamment common-test.sh indispensable).

5 Couverture des tests (JaCoCo)

5.1 Utilisation et Interprétation

Nous avons utilisé l'outil **JaCoCo** (Java Code Coverage) pour mesurer le taux de couverture de nos tests unitaires sur le code Java du compilateur.

Commande :

```
mvn jacoco:report
```

Le rapport est ensuite disponible dans `target/site/jacoco/index.html` (`mvn verify` génère également le rapport JaCoCo).

5.2 Analyse des résultats

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
fr.ensimag.deca.syntax		77%		60%	433	667	445	2,023	244	382	1	51
fr.ensimag.deca.tree		88%		82%	181	846	233	2,196	72	512	0	86
fr.ensimag.deca		81%		73%	33	111	55	296	7	50	1	6
fr.ensimag.deca.context		85%		78%	34	148	42	252	25	123	0	21
fr.ensimag.deca.codegen		94%		80%	42	185	24	431	9	97	1	10
fr.ensimag.ima.pseudocode.instructions		64%		n/a	22	62	38	111	22	62	16	54
fr.ensimag.ima.pseudocode		86%		80%	20	84	26	180	16	74	1	26
fr.ensimag.deca.tools		94%		100%	1	16	3	39	1	13	0	3
Total	3,905 of 23,686	83%	398 of 1,542	74%	766	2,119	866	5,528	396	1,313	20	257

FIGURE 2 – Rapport JaCoCo (Sans Extension RISC-V)

— **Taux de couverture global : 83 %**

— **Taux par étape :**

  — Étape A (Syntaxe) : **77 %**

  — Étape B (Contexte) : **85 %**

  — Étape C (Gencode) : **94 %**

  — Arbre : **88 %**

Analyse critique : La partie la mieux couverte est l'Analyse Contextuelle, car la logique de vérification de types se prête bien aux tests unitaires. Les parties moins couvertes concernent principalement les accesseurs simples dont le test n'était pas pertinent. L'utilisation de Jacoco nous a permis d'identifier des branches de code "mortes" ou non testées et d'ajouter des tests *Glass box* spécifiques pour combler ces lacunes.

Globalement, la couverture de notre code est très satisfaisante, bien qu'elle n'atteste en réalité pas de la qualité des tests. JaCoCo a tout de même été un outil indispensable à l'avancement des tests du code.

6 Autres méthodes de validation

En complément des tests automatisés, nous avons utilisé :

— **Revue de code :** Chaque fonctionnalité implémentée a été relue par au moins un autre membre du groupe. Cela permet de détecter des erreurs de logique ou des problèmes que les tests automatiques ne voient pas.

Appendices

A Risques Organisationnels

GL25	RISQUES ORGANISATIONNELS		
RISQUE	**PROBABLITÉ**	**GRAVITÉ**	**ACTIONS**
Retard dans l'avancement global du projet	Moyenne	Élevée	1. Découpage en Sprint pour une répartition optimale du travail 2. Réalisation d'un planning prévisionnel
Démotivation d'un membre de l'équipe	Faible	Très élevée	Donner des responsabilités et un rôle précis à chacun
Mauvaise priorisation	Moyenne	Élevée	Être attentif à ce qui est demandé dans les points de suivis ainsi que dans le rendu intermédiaire
Oubli des deadlines	Faible	Très élevée	1. Élaboration d'un planning avec marqueur sur les dates de rendus
Mauvais nommage des fichiers	Faible	Élevée	1. Respect strict des conventions données en page 27 du poly 2. Vérification avant le rendu
Dépôt Git incomplet	Faible	Très élevée	1. Se référer au document "Gestion des rendus" 2. Vérifier la conformité du git vis-à-vis des attendus du rendu
Mauvaise communication dans l'équipe	Faible	Très élevée	1. Point régulier dans l'équipe 2. Serveur Discord dédié
Déséquilibre de la charge de travail	Moyenne	Élevée	1. Répartition explicite des tâches avec l'avis de chacun 2. Communication si surcharge
Disparité de niveau trop élevée au sein de l'équipe	Moyenne	Élevée	Séances de code review pour rattraper le retard

FIGURE 3 – Risques Organisationnels

B Risques Techniques

GL25	RISQUES TECHNIQUES		
RISQUE	**PROBABLITÉ**	**GRAVITÉ**	**ACTIONS**
Le compilateur ne compile pas	Faible	Très élévée	Compiler systématiquement avant de push
Échec des tests 'common-tests.sh'	Faible	Très élevée	Lancer le script et débugger les erreurs
Bug en génération de code (Étape C)	Moyenne	Élevée	Établir des tests ciblés par fonctionnalités au fur et à mesure
Erreur d'interprétation de la grammaire (Étape B)	Moyenne	Moyenne	1. Établir des tests de contexte au fur et à mesure 2. Revoir à plusieurs le document répertoriant la gestion des erreurs
Base de tests insuffisante	Moyenne	Élevée	1. Augmentation progressive du nombre de tests et donc de la couverture 2. Établir les tests au fur et à mesure
Mauvaise classification (Valid/Invalid & Created/Provided)	Faible	Très élevée	Relecture de l'arborescence imposée avant le rendu
Non-respect de l'interface decac (options, comportement)	Faible	Très élevée	Tests systématiques des options -p, -v, decac
Messages d'erreur non conformes à la spécification	Faible	Élevée	Vérification des messages d'erreurs sur les programmes "Invalid"
Perte de données	Moyenne	Très élevée	Commits réguliers sur le git de l'équipe

FIGURE 4 – Risques Techniques